



UNIVERSITÀ DI PISA

Dipartimento di Ingegneria dell'Informazione

Laurea Triennale in Ingegneria Informatica

***Progettazione e realizzazione di una dashboard per il
monitoraggio di rete***

Relatori

Prof. Enzo Mingozzi

Prof. Antonio Virdis

Candidata

Asia Perroni

Anno accademico

2025/2026

Indice

1	Introduzione	2
1.1	Presentazione dell'argomento di tesi	2
2	Svolgimento	3
2.1	Secifiche tecniche.....	3
2.2	Strumenti	3
2.2.1	Containerlab.....	3
2.2.2	FRR	7
2.2.3	Prometheus	7
2.2.4	Exporter.....	9
2.2.5	Grafana	10
2.3	Realizzazione del sistema	10
2.3.1	Integrazione degli exporter	10
2.3.2	Topologia di rete finale	12
2.4	Risultati.....	14
2.4.1	Dashboard.....	14
2.4.2	Test	25
3	Conclusioni	33
3.1	Considerazioni.....	33
3.2	Possibili sviluppi	33
4	Bibliografia	34

1 Introduzione

1.1 Presentazione dell'argomento di tesi

Il progetto di tesi descritto in questo documento si propone di simulare uno scenario in cui un'organizzazione di medie dimensioni necessiti di costruire un sistema per monitorare la propria infrastruttura di rete mediante una dashboard.

Per organizzazioni e imprese che dispongono di una rete informatica proprietaria, monitorarne lo stato di salute favorisce la rapida intercettazione di malfunzionamenti e comportamenti indesiderati, favorendo la qualità e la continuità dei servizi offerti.

Innanzitutto è necessario individuare degli “indicatori di salute”. Molte informazioni sullo stato interno dei dispositivi possono essere tratte dall'osservazione via software dello stato delle risorse, come CPU, RAM, interfacce di rete e, nel caso dei dispositivi di rete, dei protocolli come OSPF. Monitorare questi parametri accedendo individualmente a ciascun dispositivo può essere molto dispendioso in termini di tempo. Ancora peggiore sarebbe la situazione in cui si presentasse un disservizio nella rete: l'individuazione della causa potrebbe non essere banale se non si dispone di alcuna indicazione su quali siano i dispositivi coinvolti.

Questi sono alcuni dei motivi per cui può essere utile avere i dati più rilevanti a disposizione su un'unica piattaforma, già elaborati e aggregati per mettere in risalto le informazioni di interesse. Una dashboard è un'implementazione di tale piattaforma e ha il vantaggio di presentare i dati in forma grafica, in modo che valori e andamenti anomali siano facilmente individuabili.

2 Svolgimento

2.1 Specifiche tecniche

Nella fase di definizione dei requisiti, sono state tracciate le linee guida da osservare nello sviluppo. Nello specifico il sistema avrebbe dovuto permettere:

- il monitoraggio remoto dei nodi di una rete attraverso un end-point arbitrario;
- la memorizzazione centralizzata dei dati raccolti, in modo che non impiegassero la memoria dei nodi monitorati;
- di rendere disponibili i grafici da un'interfaccia web;
- di filtrare le informazioni in base agli specifici nodi di interesse.

Con queste specifiche in mente, sono stati individuati gli strumenti che avrebbero costituito lo scheletro del sistema:

- Containerlab come ambiente di emulazione della rete;
- Free Range Routing (FRR) per implementare i nodi router;
- Software exporter, come FRR exporter, per estrarre le metriche dai dispositivi;
- Prometheus per la raccolta, la memorizzazione e l'elaborazione dei dati;
- Grafana per la creazione dei grafici e la presentazione tramite interfaccia web.

Gli strumenti scelti sono tutti open source e largamente usati e documentati.

In Figura 1 viene presentata una bozza iniziale della pipeline formata da questi strumenti.

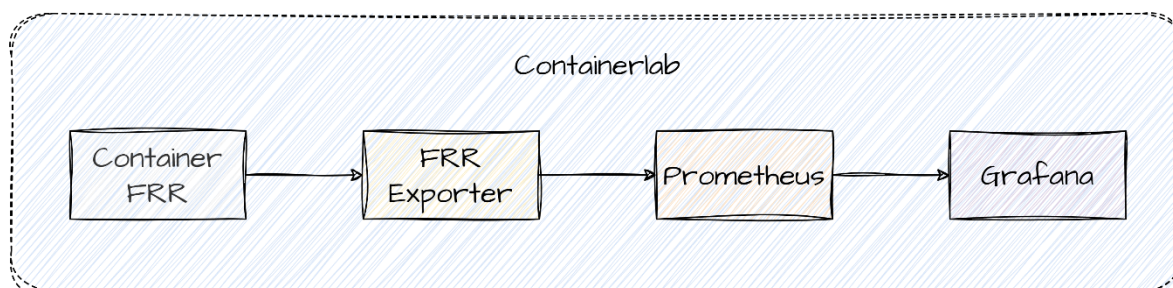


Figura 1 - Bozza iniziale della pipeline di monitoraggio

2.2 Strumenti

In questa sezione verranno presentati gli strumenti utilizzati e le feature principali adoperate per la realizzazione del progetto.

2.2.1 Containerlab

Containerlab è un software con interfaccia a linea di comando (CLI) che permette di creare e gestire reti di container Docker. Il principale requisito per eseguirlo è un

sistema operativo Linux, anche virtualizzato, su cui sia installato Docker, per questo l'ambiente è stato eseguito su WSL2. Uno dei vantaggi di Containerlab è che per simulare una rete è sufficiente un file YAML che ne descriva la topologia (*topology file*), secondo la sintassi definita nella documentazione. Un altro vantaggio sta nel fatto che, essendo basato su container Docker, l'impiego di risorse dell'host che esegue la simulazione è contenuto, rispetto ad esempio ad una simulazione basata su macchine virtuali. A differenza delle macchine virtuali, che emulano tutte le risorse di un calcolatore fisico, la tecnologia dei container sfrutta direttamente il kernel del sistema operativo dell'host, pur garantendo l'isolamento tra gli ambienti host e container.

Di seguito si propone un esempio di file di topologia, per chiarire alcuni dei concetti chiave necessari a comprendere gli aspetti implementativi della rete.

```

name: testlab          # Nome del laboratorio.

topology:
  nodes:
    router1:           # Nome del nodo
      kind: linux
      image: quay.io/frrouting/frr:10.7.1
      binds:
        - ./router1.conf:/etc/frr/frr.conf:ro

    host1:
      kind: linux
      image: alpine:latest
      exec:
        - ip addr add 192.168.0.2/24 dev eth1

  links:
    - endpoints: ["router1:eth1", "host1:eth1"]

```

- **name:** identifica il laboratorio. Containerlab genera i nomi dei container seguendo il formato `clab-${nome-lab}-${nome-nodo}`.
- **topology:** elemento che racchiude la definizione della rete.
- **nodes:** ogni chiave sotto questo elemento costituisce il nome di un nodo e viene usato come hostname del container.
- **kind:** il tipo determina la procedura che Containerlab deve svolgere per avviare un container. Ad esempio: le chiamate di sistema, le directory istanziate, l'entrypoint, i comandi da eseguire all'avvio e così via. Questi step saranno gli stessi in container Debian o Centos, basati sul tipo linux, ma diversi da quelli necessari per avviare, ad esempio, un container Cisco Nexus 9000v, per cui è necessario usare il kind `cisco_n9kv`.
- **image:** immagine Docker con cui avviare il container, che determina l'applicazione eseguita ed eventuali altri componenti installati a supporto.
- **binds:** direttiva che precede un elenco di file o directory presenti nella macchina host da rendere accessibili al container, con la sintassi

`<percorso file host>:<percorso mount container>:<modalità di accesso>`

dove la modalità di accesso indica i permessi di accesso al file che verranno assegnati ai processi istanziati nel container. Nel caso in esempio, il file `router1.conf` presente nella stessa directory del file di topologia verrà reso disponibile al container nella directory `/etc/frr` con il nome di `frr.conf` e permessi di sola scrittura (*ro*).

- **exec:** lista di comandi che il processo utente eseguirà nel terminale del container appena dopo la procedura di avvio. È importante specificare che questi comandi vengono eseguiti solo alla prima accensione, perciò se il

container viene riavviato gli effetti non persistono. Ad esempio, eseguendo il comando `docker restart clab-testlab-host1`, l'interfaccia `eth1` non manterrebbe l'indirizzo assegnato.

- **links:** lista di collegamenti punto-punto tra i nodi. La sintassi è:
“<nodo A>:<interfaccia A>”, “<nodo B>:<interfaccia B>”.

La simulazione può essere avviata tramite il comando

```
clab [global-flags] deploy [local-flags].
```

Se la directory corrente è la stessa del file di topologia e quest'ultimo è denominato con il formato <nome lab>.clab.yml, è possibile omettere i flag e Containerlab troverà automaticamente il file. Altrimenti è necessario almeno il flag
`-t /path/to/topology-file.yml`.

Una volta cominciata l'emulazione, è possibile investigare lo stato dei container con il comando `clab inspect`, che mostrerà:

- nome del container come assegnato da Containerlab;
- kind e immagine;
- stato (running, restarting, exited, ecc.);
- indirizzo IP dell'interfaccia di management (di default la `eth0` per i container con kind: linux).

È possibile eseguire comandi dentro la shell del container utilizzando:

```
docker exec <nome container> <comando>.
```

Si supponga che nel file di configurazione `router1.conf` venga assegnato l'IP `192.168.0.1` all'interfaccia `eth1` di `router1`. Di seguito viene mostrato come eseguire un ping verso quell'interfaccia mediante l'interfaccia del container `host1`.

```
clab@LAPTOP-BSFQRKKR:~/Tesi/testlab$ docker exec clab-testlab-host1
ping -c1 192.168.0.1
PING 192.168.0.1 (192.168.0.1): 56 data bytes
64 bytes from 192.168.0.1: seq=0 ttl=64 time=0.159 ms

--- 192.168.0.1 ping statistics ---
1 packets transmitted, 1 packets received, 0% packet loss
round-trip min/avg/max = 0.159/0.159/0.159 ms
```

È anche possibile avviare una sessione interattiva nel terminale del container con il comando

```
docker exec -it clab-testlab-host1 ash,
```

dove i flag `-it` fanno sì che venga aperta una sessione interattiva con il terminale del container, che può essere chiusa eseguendo il comando `exit`. `ash` è la shell disponibile nei container di tipo Linux.

2.2.2 FRR

Free Routing Range (FRR) è un software open source supportato da tutti i moderni sistemi operativi NIX. Offre servizi di routing IP e implementa tutti i protocolli di routing standard. Gestisce il piano di controllo e installa le route nel kernel, che si occupa poi del forwarding di pacchetti.

FRR è un software modulare: ogni protocollo di routing disponibile è implementato da un *daemon*. Questi sono messi in comunicazione da un ulteriore daemon, zebra, che si interfaccia con il piano dati (Figura 2). È possibile abilitare o disabilitare ogni daemon tramite un unico file di configurazione. L'architettura modulare rende possibile aggiungere moduli per protocolli non disponibili nel pacchetto di default. Questo aspetto è stato cruciale nel progetto, in quanto ha permesso di estrapolare un gran numero di metriche usate nella dashboard tramite Simple Network Management Protocol (SNMP), non integrato nativamente in FRR.

Tutti i daemon FRR possono essere gestiti tramite una singola shell, *vtys*, che fornisce un set di comandi *Cisco-like* e permette di configurare i daemon tramite un unico file di configurazione.

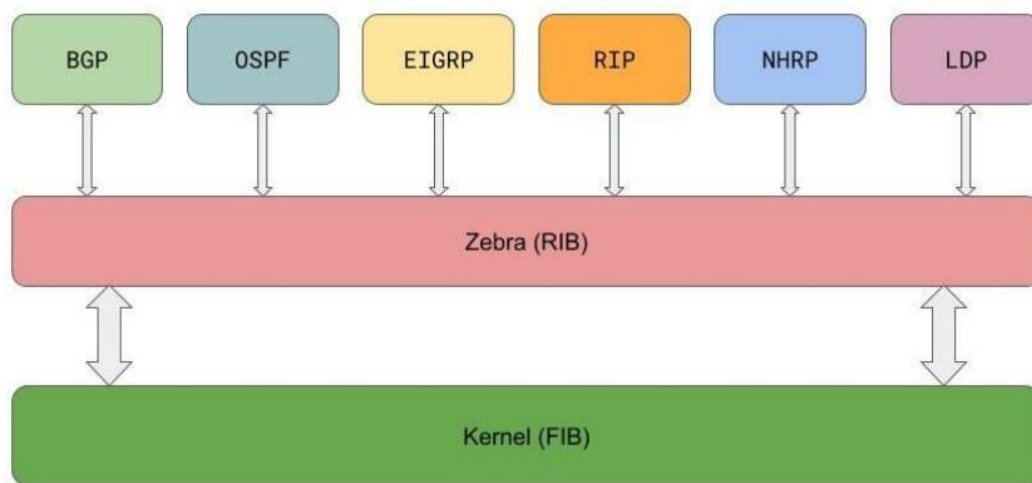


Figura 2 - Diagramma dell'architettura di FRR. Fonte: <https://www.nextplatform.com/connect/2020/10/26/frr-the-most-popular-network-router-youve-never-heard-of/1636827>.

2.2.3 Prometheus

Prometheus è un insieme di strumenti software open-source impiegato per realizzare sistemi di monitoraggio. In particolare si occupa di raccogliere dagli exporter i dati (*metriche*) riguardanti i dispositivi target, conservarli e renderne possibile la consultazione e l'elaborazione.

Prometheus raccoglie dati ad intervalli di tempo regolari e configurabili, per ogni metrica viene quindi creata una serie temporale (o *time serie*), ovvero una serie ordinata di coppie [*numero reale*, *istante*] che permette di osservare la variazione della misura

nel tempo. Nel modello dati di Prometheus, ogni time serie è identificata univocamente dal nome della metrica e da una serie di coppie [*chiave*, *valore*], in cui la chiave assume il nome di *etichetta* (o *label*), secondo la seguente notazione:

```
<nome metrica>{<nome etichetta>="<valore etichetta>", ...}
```

Si prenda, ad esempio, una generica metrica `api_http_requests_total` che misura il conteggio totale delle richieste HTTP ricevute dall'API di un'ipotetica applicazione web. Si supponga che le time serie per questa metrica presentino le etichette `method` e `handler`. A seconda del valore di queste, si individuano time serie diverse come, ad esempio:

```
api_http_requests_total{method="POST", handler="/messages"}
```

```
api_http_requests_total{method="GET", handler="/home"}
```

```
api_http_requests_total{method="POST", handler="/login"}.
```

Prometheus salva le time series in un Time Series Data Base (TSDB), un database ottimizzato per la memorizzazione e la lettura di questo tipo di dati. È possibile selezionare questi dati, elaborarli e aggregarli tramite query PromQL (Prometheus Query Language). In Figura 3 è riportato uno schema semplificato dell'architettura prometheus.

Le query possono essere eseguite da un servizio esterno tramite API HTTP, da linea di comando o dall'interfaccia web che il server Prometheus espone di default alla porta 9090.

Una semplice query è la selezione di una metrica. Ad esempio la query

```
api_http_requests_total
```

restituisce un vettore di interi. Ogni intero è associato a valori diversi per le label `method` e `handler`. Una query simile si definisce *instant query* e il vettore risultato *instant vector*, perché gli elementi del vettore sono tutti associati allo stesso timestamp, di default il più recente.

È possibile filtrare la selezione specificando dei valori per una o più etichette. Ad esempio `api_http_requests_total { method = "GET" }` restituisce solo i contatori per le request avvenute con metodo GET.

Se in aggiunta alla metrica si specifica anche un intervallo temporale, ad esempio 5 minuti, si esegue invece una *range query*, che restituisce un *range vector*, in cui ad ogni combinazione di label sono associate tutte le misurazioni avvenute nei 5 minuti precedenti all'esecuzione della query. Un esempio di range query è:

```
api_http_requests_total { method = "GET" } [5m].
```

La configurazione di Prometheus avviene tramite un file YAML. In esso vengono specificati, tra le altre cose, gli endpoint da monitorare (target), l'intervallo con cui

raccogliere le metriche (scrape interval) e le eventuali regole di elaborazione dei dati. I target sono organizzati in gruppi detti job, ciascuno dei quali può avere parametri di raccolta specifici.

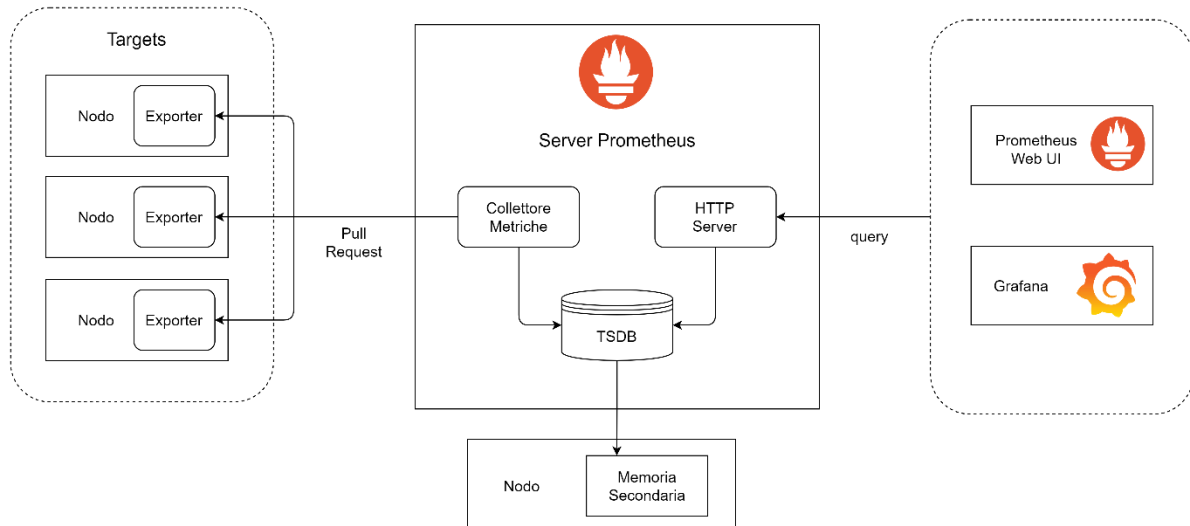


Figura 3 - Diagramma dell'architettura di Prometheus.

2.2.4 Exporter

In questo paragrafo verrà fatta una breve descrizione del funzionamento degli exporter, tralasciando i dettagli su come questi leggono le metriche dal sistema monitorato, in quanto variabili a seconda della metrica e dello specifico exporter e poco rilevanti ai fini della realizzazione del progetto di tesi.

Un exporter è un software che legge dati operativi da un sistema (sistema operativo, database, hardware, ecc.) e li elabora per fornire delle metriche a software terzi. La sintassi delle metriche ed il procedimento con cui queste vengono trasferite dipende principalmente da quale sia il software destinato a raccoglierle e conservarle. Nel caso di exporter compatibili con Prometheus, come quelli usati nel presente progetto, queste metriche vengono esposte ad un endpoint HTTP, in un formato compatibile con Prometheus. Il paradigma di trasferimento è di tipo *pull*, quindi Prometheus inizializza una connessione TCP ed invia una richiesta HTTP sulla porta a cui sono esposte le metriche, solitamente una nel range 1024 – 49151, ad un endpoint prestabilito, solitamente */metrics*. Di norma ogni exporter utilizza una porta di default nota ma se necessario configurabile. In gergo Prometheus, il processo di raccolta delle metriche da un exporter viene indicato con il termine *scraping*¹.

¹ Il verbo "to scrape" può essere tradotto letteralmente con il termine *raschiare*.

2.2.5 Grafana

Grafana è una piattaforma open-source per la visualizzazione e l'analisi di dati provenienti da diverse sorgenti. Nel contesto dei sistemi di monitoraggio viene comunemente utilizzata insieme a Prometheus, dal quale recupera le metriche mediante query e le presenta sotto forma di grafici, tabelle e altri elementi visuali raccolti in dashboard interattive. Grafana supporta inoltre la parametrizzazione delle dashboard tramite variabili, consentendo di adattare dinamicamente la visualizzazione ai dati selezionati dall'utente.

Nel progetto realizzato, Grafana è stato impiegato per costruire dashboard dedicate al monitoraggio dei router e dei protocolli di routing. Per migliorare l'organizzazione delle informazioni, i pannelli sono stati raggruppati mediante row e tab, mentre l'utilizzo di variabili ha consentito di selezionare dinamicamente router, aree OSPF e altre entità monitorate. Infine, le trasformazioni messe a disposizione da Grafana sono state utilizzate per riorganizzare e aggregare i dati restituiti dalle query Prometheus, in particolare per la realizzazione di tabelle riepilogative e matrici di dati.

2.3 Realizzazione del sistema

Il primo obiettivo del progetto è stato la costruzione della pipeline di monitoraggio. Una volta individuati gli strumenti software da utilizzare, è stato realizzato un laboratorio di test mediante Containerlab.

Per i test iniziali è stata utilizzata una topologia semplice, costituita da due router FRR collegati tramite due sottoreti distinte e configurati all'interno della stessa area OSPF. Tale ambiente è stato utilizzato per verificare il corretto funzionamento della raccolta delle metriche prima di passare a scenari più complessi.

Nel seguito il termine *host*, verrà usato per indicare l'ambiente locale WSL utilizzato per eseguire il laboratorio.

2.3.1 Integrazione degli exporter

Il primo passo dell'integrazione degli exporter è stato l'esplorazione delle metriche disponibili e della loro struttura. Per questa attività non è stato necessario utilizzare Prometheus: tutti gli exporter espongono infatti le metriche tramite endpoint HTTP, accessibili direttamente dall'host.

Containerlab consente di pubblicare porte dei container sull'host mediante la direttiva `ports` del file di topologia, permettendo così di consultare le metriche direttamente tramite browser o strumenti da linea di comando.

2.3.1.1 FRR Exporter

FRR Exporter è un exporter dedicato al monitoraggio dei router FRR.

L'exporter è disponibile sia come immagine container sia come sorgente compilabile. Poiché le metriche vengono ottenute leggendo i socket Unix utilizzati internamente dai processi FRR, si è scelto di compilare l'eseguibile localmente e renderlo disponibile ai container tramite bind mount definito nel file di topologia.

FRR Exporter espone metriche relative alla maggior parte dei protocolli supportati da FRR, tra cui BGP e OSPF. Nell'ambito del progetto sono state utilizzate principalmente le seguenti informazioni:

- numero di vicini OSPF;
- stato delle adiacenze OSPF;
- identificazione dei vicini tramite Router ID e indirizzo IP dell'interfaccia;
- composizione quantitativa del Link State Database (LSDB) per area;
- numero di rotte presenti nella Routing Information Base (RIB);
- numero di rotte presenti nella Forwarding Information Base (FIB);
- distribuzione delle rotte per tipologia (OSPF, statiche, connected e locali).

2.3.1.2 SNMP Exporter

Per ampliare il numero di informazioni disponibili è stato integrato anche SNMP Exporter.

Il protocollo SNMP

Di seguito si fa una breve introduzione al Simple Network Management Protocol (SNMP), ritenuta necessaria a comprendere come l'exporter viene integrato nel sistema. SNMP è un protocollo di management dello Standard Internet. È usato per raccogliere e organizzare informazioni riguardo lo stato del sistema e le configurazioni di dispositivi nelle reti IP. I dati raccolti da SNMP sono esposti sotto forma di variabili, univocamente identificabili tramite *object identifier* (OID). Tali variabili sono organizzate gerarchicamente all'interno di strutture denominate Management Information Base (MIB).

Un sistema gestito con SNMP si compone di:

- uno o più dispositivi monitorati, detti *managed object*;
- un software di gestione, o *agente*, eseguito dai dispositivi monitorati;
- un sistema di gestione, anche detto *manager*, che interroga gli agenti.

Integrazione nel laboratorio

SNMP Exporter è stato distribuito come container dedicato e configurato per operare come manager SNMP.

Nel contesto SNMP, i router FRR svolgono il ruolo di agenti. Poiché FRR non implementa direttamente un agente SNMP, è stato necessario installare il pacchetto `snmpd`, che fornisce il daemon incaricato di rispondere alle richieste SNMP.

Un'ulteriore difficoltà è stata rappresentata dal fatto che il supporto SNMP in FRR deve essere abilitato in fase di compilazione. Le immagini ufficiali di FRR non includono tale supporto, rendendo necessaria la creazione di un'immagine personalizzata mediante modifica del Dockerfile.

Metriche raccolte

Le informazioni ottenibili dipendono dagli OID configurati nel file YAML di SNMP Exporter.

Nel progetto, SNMP Exporter è stato utilizzato principalmente per integrare le informazioni fornite da FRR Exporter relativamente al protocollo OSPF. In particolare sono state raccolte metriche riguardanti:

- numero di LSA trasmessi e ricevuti;
- associazione tra vicini OSPF e area di appartenenza;
- ruolo assunto dal router all'interno della rete (ABR, ASBR, DR e BDR);
- ulteriori contatori e statistiche previsti dalla MIB OSPF.

2.3.1.3 cAdvisor

cAdvisor (Container Advisor) è un exporter sviluppato specificamente per il monitoraggio dei container.

A differenza di FRR Exporter e SNMP Exporter, che raccolgono informazioni relative ai protocolli di rete e allo stato dei router, cAdvisor fornisce metriche sull'utilizzo delle risorse computazionali da parte dei container. Tra queste rientrano:

- utilizzo della CPU;
- consumo di memoria;
- traffico di rete;
- statistiche sui processi;
- utilizzo del filesystem.

L'integrazione di cAdvisor ha consentito di monitorare non solo il comportamento della rete emulata, ma anche il consumo di risorse dell'infrastruttura di laboratorio stessa.

2.3.2 Topologia di rete finale

Una volta verificato il corretto funzionamento della pipeline di monitoraggio e degli exporter nel laboratorio iniziale, si è proceduto alla realizzazione di uno scenario più

rappresentativo di una rete reale. L'obiettivo era valutare il comportamento del sistema di monitoraggio in presenza di una topologia più articolata e di un protocollo di routing distribuito su più aree.

Lo schema di rete è riportato in Figura 4 e si compone di sei router organizzati in tre aree OSPF: l'area 0 svolge il ruolo di backbone e connette due aree periferiche, ciascuna composta da una coppia di router. I container di monitoraggio, nel diagramma schematizzati con icone server, sono stati collocati nella medesima sottorete nell'area 1, collegata tramite bridge all'interfaccia di uno dei router FRR.

A differenza della configurazione iniziale, i router FRR sono stati avviati con l'opzione `network-mode: none`, escludendoli dalla rete di management generata automaticamente da Containerlab. In questo modo il traffico di routing e monitoraggio transita esclusivamente attraverso i collegamenti definiti nella topologia, consentendo di osservare e analizzare il comportamento della rete senza l'influenza della rete di management utilizzata dall'host. I servizi di monitoraggio, invece, sono rimasti connessi alla rete host, così da mantenere accessibili le interfacce web di Prometheus e Grafana durante le attività di sviluppo e analisi.

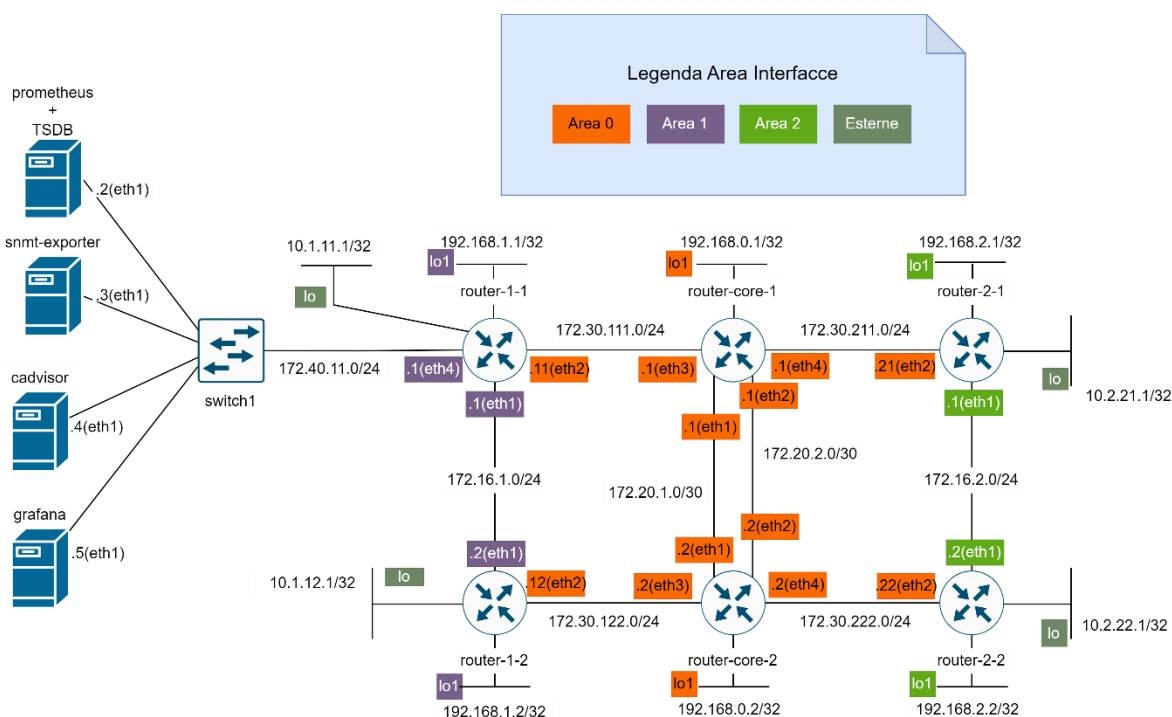


Figura 4 - Schema della rete utilizzata nella versione finale del progetto.

Hostname	Router ID OSPF
router-core-1	1.1.1.1
router-core-2	2.2.2.2
router-1-1	3.3.3.3
router-1-2	4.4.4.4

router-2-1	5.5.5.5
router-2-2	6.6.6.6

Tabella 1 - ID OSPF assegnati ai router.

2.4 Risultati

La dashboard sviluppata ha come focus il monitoraggio dei router. Viene analizzato il networking a livello di routing, con attenzione al protocollo OSPF, le prestazioni di rete e l'utilizzo di risorse computazionali da parte di ciascun router. Inoltre, sono stati aggiunti panel che forniscono una panoramica del corretto funzionamento degli stessi strumenti di monitoraggio.

2.4.1 Dashboard

2.4.1.1 Tab: Stato Strumenti di Monitoraggio

Perché il sistema di monitoraggio funzioni, tutte le sue componenti devono essere operative. In questa sezione, mostrata in Figura 5, se ne monitora lo stato di servizio.



Figura 5 - Sezione con vista riassuntiva dello stato di salute degli strumenti di monitoraggio.

Panel	Stato Container di Monitoraggio
Query	<pre>up{job="prometheus"}</pre> <pre>up{job="cadvisor"}</pre> <pre>up{job="snmp-exporter"}</pre>
Descrizione	Panoramica dello stato dei container di monitoraggio centralizzati. Valori possibili: UP, DOWN.

<i>Panel</i>	Monitoraggio Router
<i>Query</i>	<pre>label_replace(sysName, "hostname", "\$1", "sysName", "(.*)") * on (instance) group_left up{job="snmp- network-devices"} up{job="frr"} frr_collector_up{collector="ospf"} frr_collector_up{collector="route"} frr_collector_up{collector="status"}</pre>
<i>Descrizione</i>	Panoramica dello stato degli exporter, o delle loro componenti, installati in ciascun router. SNMP Exporter: indica se l'exporter è in grado di raggiungere l'agent SNMP installato nel router; FRR Exporter: indica se il processo è attivo nel router; collector: stato dei collector attivati di FRR Exporter. Valori possibili: UP, DOWN.

<i>Panel</i>	Utilizzo CPU FRR Exporter
<i>Query</i>	<pre>rate(process_cpu_seconds_total{job="frr"} [5m])*100</pre>
<i>Descrizione</i>	Visualizza la percentuale di utilizzo della CPU dei router da parte di FRR Exporter. Il valore è calcolato come rate della metrica, di tipo counter, che misura in secondi il tempo totale di CPU dedicato al processo.

2.4.1.2 Tab: Risorse

In questa sezione viene monitorato lo stato di utilizzo delle risorse computazionali. La sezione è divisa in due sottosezioni: la prima relativa agli strumenti di monitoraggio, la seconda ai router. Tutte le metriche di questa sezione sono state prelevate utilizzando cAdvisor. Nella prima sezione (Figura 6), è possibile filtrare di quali strumenti si vogliono visualizzare i dati tramite la variabile *Strumento*, riferita nelle query come *\$strumento*.

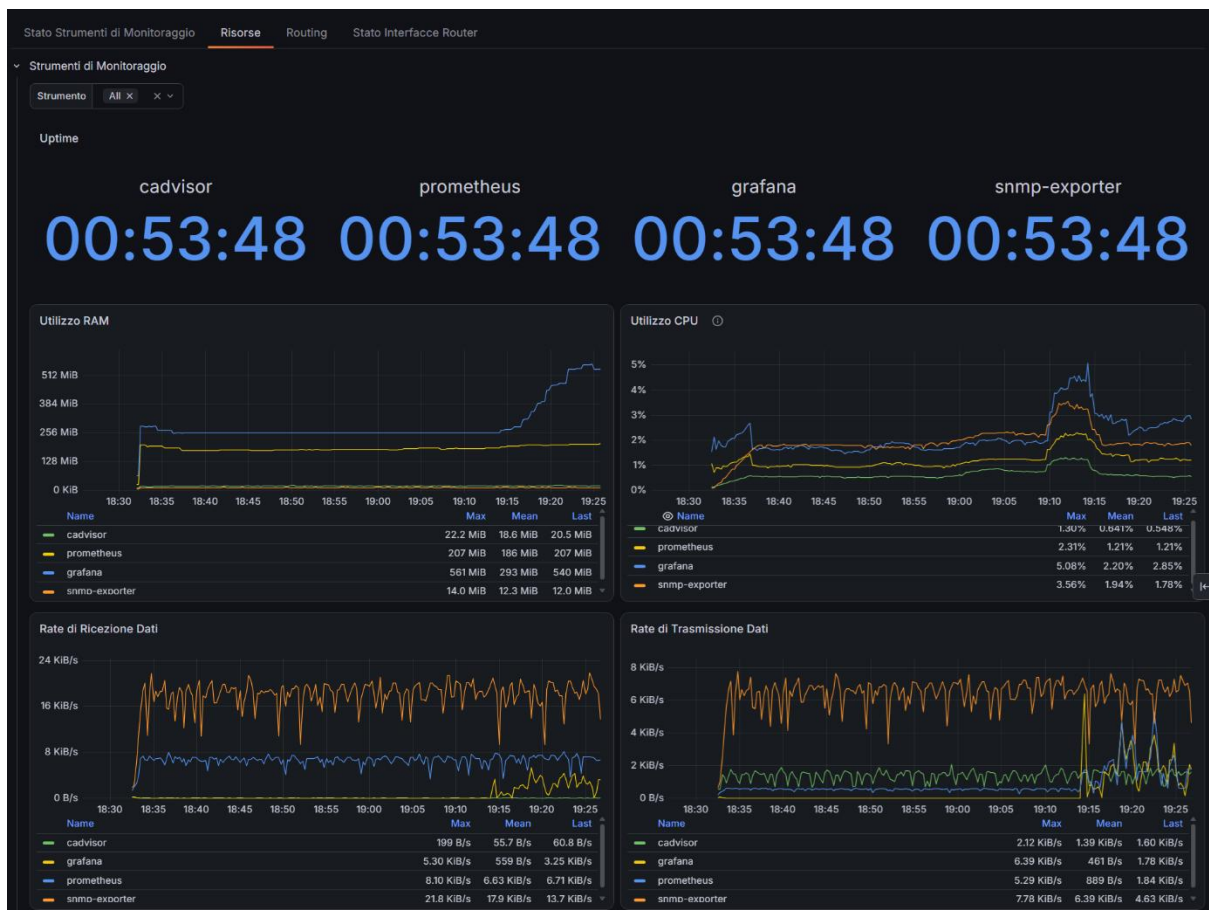


Figura 6 - Sezione utilizzo risorse dedicata agli gli strumenti di monitoraggio.

Panel	Uptime
Query	<pre>(time() - container_start_time_seconds{name=~"clab-[a-zA-Z0-9]+-\$strumento"})</pre> or <pre>(container_last_seen{name=~"clab-[a-zA-Z0-9]+-\$strumento"} * 0)</pre>
Descrizione	In caso di container attivo riporta il tempo trascorso dall'avvio, in formato <i>ore:minuti:secondi</i> . Altrimenti, la scritta <i>Inattivo</i> .
Note sulla query	Generalmente il tempo nelle metriche e funzioni Prometheus è esposto in formato timestamp Unix². <code>time()</code> restituisce il tempo corrente, <code>container_start_time_seconds</code> il timestamp di avvio del container. Normalmente, se un container non è attivo, una instant query su un suo parametro non restituisce alcun parametro; quindi, in quel caso, la statistica relativa allo strumento non verrebbe affatto visualizzata. Leggendo le sue metriche direttamente dall'host, cAdvisor è in grado di rilevare anche i container non più attivi. La sua metrica

² Numero di secondi trascorsi dal 1° gennaio 1970 alle 00:00:00 UTC.

`container_last_seen` restituisce il timestamp dell'ultima volta che un determinato container è stato in funzione.
 “`or (container_last_seen{...} * 0)`” è un espediente per far sì che il panel relativo ad un container inattivo rimanga visibile. In quel caso, il valore riportato è messo a 0, e mappato a “Inattivo” nelle opzioni del panel Grafana.

Panel	Utilizzo RAM
Query	<code>container_memory_usage_bytes{name=~"(.*)\$strumento\$"}/1024</code>
Descrizione	Memoria utilizzata dal processo, espressa in MiB.

Panel	Utilizzo CPU
Query	<code>rate(container_cpu_usage_seconds_total{name=~"(.*)\$strumento\$"}[5m]) * 100</code>
Descrizione	Percentuale di CPU utilizzata dal processo, ottenuta calcolando il rate su un'intervallo di 5 minuti

Panel	Rate di Ricezione Dati
Query	<code>sum by (name) (rate(container_network_receive_bytes_total{name=~"(.*)\$strumento\$"}[\$__rate_interval]))</code>
Descrizione	Rate su di byte in ingresso alla NIC del container. Particolarmente rilevante nel caso di Prometheus, in quanto un numero elevato di target unito ad un'alta cardinalità nelle label possono causare un <i>burst</i> nell'utilizzo di banda a causa di un'elevato numero di metriche ricevuto dagli exporter.
Note sulla query	Nella rete emulata da Containerlab, i container hanno due interfacce: la <code>eth1</code> collegata tramite bridge a router-1-1 e l'interfaccia di management <code>eth0</code> tramite cui il container può comunicare direttamente con l'host. Nel caso dei container di monitoraggio, si è deciso di trascurare questa separazione, da cui l'aggregazione tramite <code>sum by (name)</code> . <code>\$__rate_interval</code> è una variabile globale di Grafana, da utilizzare nelle funzioni <code>rate()</code> e <code>increase()</code> , che viene sostituita con una finestra di tempo sufficientemente ampia da racchiudere quattro sample.

Panel	Rate di Trasmissione Dati
-------	---------------------------

<i>Query</i>	sum by (name) (rate(container_network_transmit_bytes_total {name=~"(.*)\$strumento\$"}[\$__rate_interval]))
<i>Descrizione</i>	Duale del panel precedente per il rate di trasmissione dati.

In Figura 7 sono mostrati i panel della seconda sottosezione, dedicata ai router. Uptime, Utilizzo CPU e Utilizzo RAM sono del tutto equivalenti alla sezione precedente. Per quanto riguarda i router, osservare il rate in trasmissione e ricezione di ciascuna interfaccia può fornire molte informazioni sul bilanciamento del traffico e sul suo flusso. In questa sottosezione, ciascun router ha la sua coppia di panel dedicata, dove si raffigura una serie temporale per ogni interfaccia.



Figura 7 - Sezione utilizzo risorse dedicata ai router.

2.4.1.3 Tab: Routing

In questa sezione vengono raccolte informazioni sulle rotte disponibili, sulle configurazioni OSPF dei singoli router e sullo stato globale del protocollo visto nell'insieme dell'autonomous system.

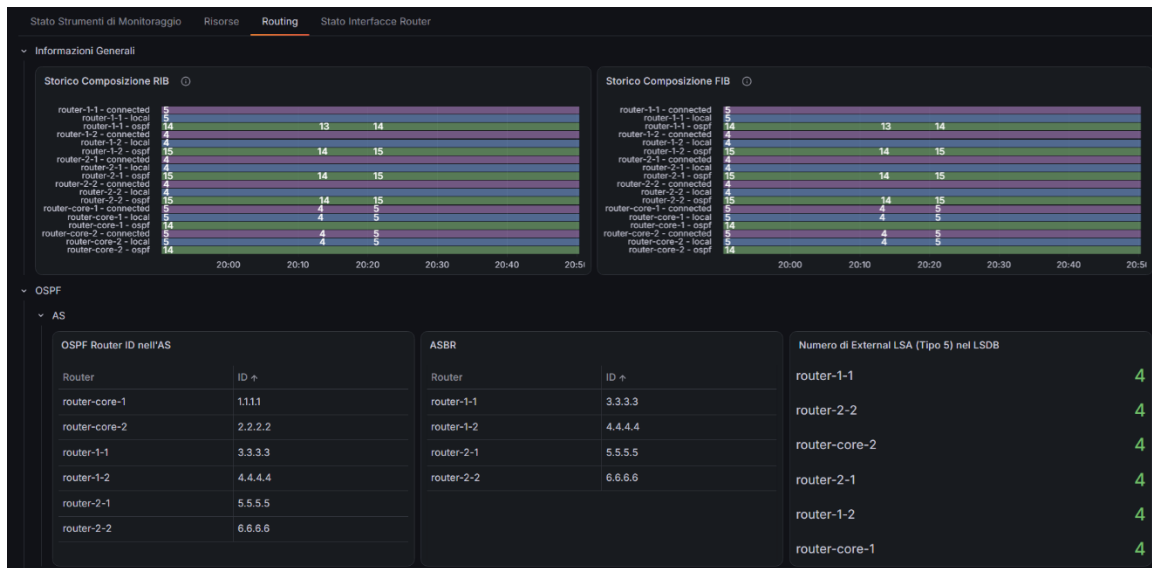


Figura 8 - Sezione dedicata al routing: composizione RIB e FIB e panoramica OSPF nell'AS

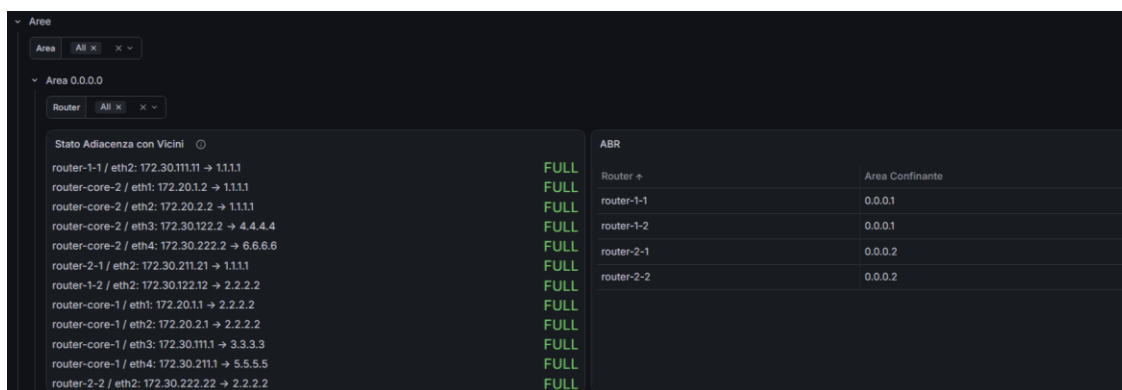
Panel	Storico Composizione RIB
Query	<code>frr_route_rib_count {route_type=~"ospf static connected local", afi="ipv4"}</code>
Descrizione	State timeline che mostra, per ciascun tipo, il numero di rotte IPv4 memorizzate nella tabella di routing. L'intento è di fornire uno strumento diagnostico: permette di rilevare visivamente se un'interfaccia presenta problemi di <i>flickering</i> ; per fare un altro esempio, può essere utile dopo un riavvio o una riconfigurazione di un dispositivo, per confrontare se il numero di rotte rilevate è consistente con quello che memorizzava in precedenza.
Panel	Storico Composizione FIB
Query	<code>frr_route_fib_count{vrf="default", route_type=~"ospf static connected local", afi="ipv4"}</code>
Descrizione	Simile al panel precedente, ma con focus sulle rotte effettivamente installate nel kernel. In una rete come quella emulata, in cui è configurato un unico protocollo di routing, una discrepanza tra le due può essere sintomo di anomalie nell'installazione delle rotte.
Panel	OSPF Router ID nell'AS
Query	<code>ospfRouterId</code>

<i>Descrizione</i>	Tabella che elenca tutti i router abilitati ad OSPF nell'AS e li associa ai corrispettivi router ID.
--------------------	--

Panel	ASBR
<i>Query</i>	(ospfASBdrRtrStatus == 1) * on(instance) group_right ospfRouterId
<i>Descrizione</i>	Tabella che elenca gli ASBR nell'AS e i rispettivi router ID.

Panel	Numero di External LSA (Tipo 5) nel LSDB
<i>Query</i>	ospfRouterId * on(instance) group_left ospfExternLsaCount
<i>Descrizione</i>	Gli External LSA rappresentano le rotte redistribute da ASBR nel dominio OSPF. In condizioni normali il loro numero dovrebbe essere coerente tra i router. Differenze tra i valori possono indicare problemi di flooding, raggiungibilità degli ASBR o sincronizzazione del LSDB.

I successivi panel della sezione sono specifici e ripetuti per ciascuna area. È possibile filtrare le aree di interesse tramite la variabile *Area* e, all'interno di esse, è possibile scegliere di quali router visualizzare le informazioni, tramite la variabile *Router*.



Stato Adiacenza con Vicini		ABR	
router-1-1 / eth2: 172.30.111.11 → 1.1.1.1	FULL	Router →	Area Confinante
router-core-2 / eth1: 172.20.1.2 → 1.1.1.1	FULL	router-1-1	0.0.0.1
router-core-2 / eth2: 172.20.2.2 → 1.1.1.1	FULL	router-1-2	0.0.0.1
router-core-2 / eth3: 172.30.122.2 → 4.4.4.4	FULL	router-2-1	0.0.0.2
router-core-2 / eth4: 172.30.222.2 → 6.6.6.6	FULL	router-2-2	0.0.0.2
router-2-1 / eth2: 172.30.211.21 → 1.1.1.1	FULL		
router-1-2 / eth2: 172.30.122.12 → 2.2.2.2	FULL		
router-core-1 / eth1: 172.20.1.1 → 2.2.2.2	FULL		
router-core-1 / eth2: 172.20.2.1 → 2.2.2.2	FULL		
router-core-1 / eth3: 172.30.111.1 → 3.3.3.3	FULL		
router-core-1 / eth4: 172.30.211.1 → 5.5.5.5	FULL		
router-2-2 / eth2: 172.30.222.22 → 2.2.2.2	FULL		

Figura 9 - Panel per OSPF specifici per area: stato delle adiacenze con vicini e ABR con rispettive aree confinanti.

Panel	Stato Adiacenza con Vicini
<i>Query</i>	ospf_neighbor_state:ospf_area{area=~"\$area_id", hostname=~"\$ospf_router"}
<i>Descrizione</i>	Mostra lo stato di adiacenza con i router vicini (indicati tramite router ID), specificando l'interfaccia, con il rispettivo indirizzo IP, collegata a quel vicino. I valori possibili sono: DOWN, INIT, 2-WAY, EX.START, EXCHANGE, LOADING, FULL. È importante tenere a mente che, nei link

broadcast, i router formano un'adiacenza completa solo con il Designated Router (BR) ed il Backup Designated Router (BDR). Gli altri router con interfacce sulla stessa rete rimangono tra loro in stato 2-WAY. La permanenza in tale stato non è quindi da considerarsi in assoluto dovuta a malfunzionamenti o errori di configurazioni. Nella rete qui emulata, per semplicità, questo caso non è contemplato e tutte le connessioni tra router sono dirette.

Panel	ABR
Query	<code>ospfAreaId{ospfAreaId=~"\$area_id", hostname=~"\$ospf_router"} * on(instance) group_right() label_replace(ospfAreaId{ospfAreaId!~"\$area_id"}, "neighboring_area", "\$1", "ospfAreaId", "(.*)")</code>
Descrizione	Elenco degli ABR nell'area, con la rispettiva area di confine.

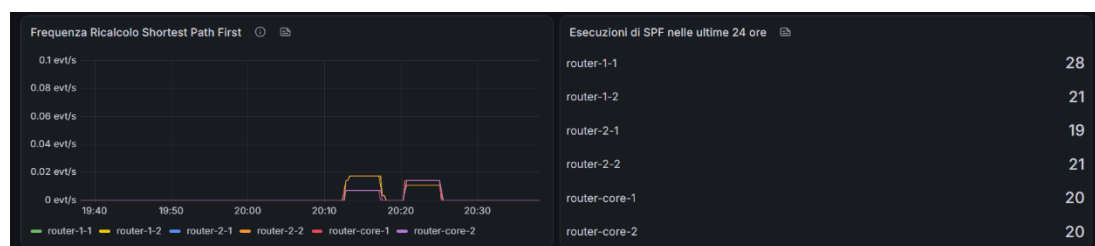


Figura 10 - Frequenza ricalcolo SPF e numero di esecuzioni dell'algoritmo nell'area 0.

Panel	Frequenza Ricalcolo Shortest Path First
Query	<code>rate(ospfSpfRuns{ospfAreaId=~"\$area_id", hostname=~"\$ospf_router"}[5m])</code>
Descrizione	Mostra il rate, misurato come eventi al secondo, con cui viene eseguito il ricalcolo della topologia di rete dell'area.

Panel	Esecuzioni di SPF nelle ultime 24 ore
Query	<code>round(increase(ospfSpfRuns{ospfAreaId=~"\$area_id"}[1d]))</code>
Descrizione	Questo panel può essere significativo in reti con una topologia stabile, rete in cui di rado vengono aggiunti o rimossi dispositivi, interfacce e

link. In tali casi, infatti, numeri alti nel contatore sono sintomo di instabilità di rete.



Figura 11 - Rate di ricezione LSA per router nell'area 0.

Panel	Rate Ricezione LS
Query	<code>ospf_ls_rate_over_5m:per_area:per_type:per_router {ospfLsdbAreaId=~"\$area_id", hostname=~"\$ospf_router"}</code>
Descrizione	In condizioni di stabilità, in cui non ci siano cambi di topologia, nuovi LSA dovrebbero essere rigenerati ogni 30 minuti (tempo di default). Picchi nel rate di ricezione possono indicare un'instabilità nella topologia.

Composizione LSDB Area 0.0.0.0					
Router +	Router LSA (Type 1) Count	Network LSA (Type 2) Count	ABR Summary LSA (Type 3) Count	ASBR Summary LSA (Type 4) Count	Totale
router-1-1	6	4	14	4	28
router-1-2	6	4	14	4	28
router-2-1	6	4	14	4	28
router-2-2	6	4	14	4	28
router-core-1	6	4	14	4	28
router-core-2	6	4	14	4	28

Figura 12 - Composizione del LSDB per ogni router nell'area 0.

Panel	Composizione LSDB Area
Query	<code># tipo 1 sum by(hostname, ospfLsdbAreaId) (ospfLsdbAreaId</code>

```

{ospfLsdbAreaId=~"$area_id", ospfLsdbType="1",
hostname=~"$ospf_router"} or
label_replace(
ospfAreaId{hostname=~"$ospf_router",
ospfAreaId=~"$area_id"}, "ospfLsdbAreaId", "$1",
"ospfAreaId", "(.*)")
* 0)

# tipo 2
sum by(hostname, ospfLsdbAreaId) (ospfLsdbAreaId
{ospfLsdbAreaId=~"$area_id", ospfLsdbType="2",
hostname=~"$ospf_router"} or
label_replace(
ospfAreaId{hostname=~"$ospf_router",
ospfAreaId=~"$area_id"}, "ospfLsdbAreaId", "$1",
"ospfAreaId", "(.*)")
* 0)

# tipo 3
sum by(hostname, ospfLsdbAreaId) (ospfLsdbAreaId
{ospfLsdbAreaId=~"$area_id", ospfLsdbType="3",
hostname=~"$ospf_router"} or label_replace(
ospfAreaId{hostname=~"$ospf_router",
ospfAreaId=~"$area_id"}, "ospfLsdbAreaId", "$1",
"ospfAreaId", "(.*)")
* 0)

# tipo 4
sum by(hostname, ospfLsdbAreaId) (ospfLsdbAreaId
{ospfLsdbAreaId=~"$area_id", ospfLsdbType="4",
hostname=~"$ospf_router"} or label_replace(
ospfAreaId{hostname=~"$ospf_router",
ospfAreaId=~"$area_id"}, "ospfLsdbAreaId", "$1",
"ospfAreaId", "(.*)")
* 0)

# totale
sum by(hostname, ospfLsdbAreaId) (ospfLsdbAreaId
{ospfLsdbAreaId=~"$area_id", hostname=~"$ospf_router"}
or label_replace(
ospfAreaId{hostname=~"$ospf_router",
ospfAreaId=~"$area_id"}, "ospfLsdbAreaId", "$1",
"ospfAreaId", "(.*)") * 0)

```

Descrizione Di norma, il LSDB relativo ad un'area di tutti i router che vi hanno un'interfaccia, dovrebbe essere identico. Questo panel dà un'informazione quantitativa, utile per cogliere a colpo d'occhio delle discrepanze tra gli LSDB.

2.4.1.4 Tab: Stato interfacce router

Questa sezione fornisce una semplice ma efficace vista dello stato amministrativo e operativo di ciascuna interfaccia di ogni router. È possibile selezionare quali router filtrare. I valori sono ottenuti mediante metriche SNMP `ifAdminStatus` e `ifOperStatus`.

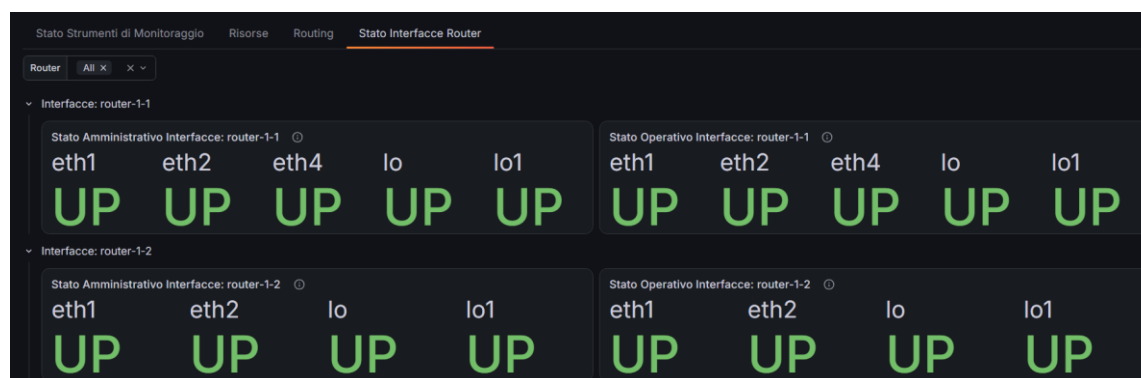


Figura 13 - Stato amministrativo e operativo delle interfacce dei router della rete. Esempio con router-1-1 e router-1-2.

2.4.2 Test

Durante la progettazione e la realizzazione del sistema, sono stati svolti dei test per verificare che determinati cambiamenti nei dispositivi producessero effetti rilevabili nella dashboard. Di seguito vengono documentati i test ritenuti più significativi, effettuati sul sistema nel suo stato finale. Per brevità, ogni test che può essere ripetuto su più dispositivi, verrà illustrato per un singolo dispositivo. Si riportano i seguenti test:

1. Disattivazione di un'interfaccia;
2. Mismatch dell'hello timer;

2.4.2.1 Test 1: disattivazione di un'interfaccia

Descrizione	Un'interfaccia del router viene spenta tramite terminale vtysh.
Modalità di esecuzione	router-core-2 : spegnimento dell'interfaccia eth3 da terminale vtysh con il comando shutdown.
Principali effetti attesi	• router-1-2 e router-core-2 perdono reciprocamente l'adiacenza; • Router con interfacce nell'area 0: ricalcolo SPF; invio e ricezione di nuovi Network LSA (tipo 2); cambiamento del LSDB; • Tutti i router: perdita di una route OSPF nella tabella di routing;

	• router direttamente connessi a router-core-2: cambiamenti del rate di ricezione e trasmissione.
--	--

Principali risultati osservati

1. Cambiamento dello stato delle interfacce ai due capi del link.

Come mostrato in Figura 14, prevedibilmente l'interfaccia eth2 di router-1-2, mantiene lo stato amministrativo UP. Lo stato operativo passa a DOWN. Ciò avviene perché l'interfaccia è stata attivata, ma non è in grado di stabilire una comunicazione con quella all'altro capo.

Interface: router-1-2

Stato Amministrativo Interface: router-1-2

eth1

UP

eth2

UP

lo

UP

lo1

UP

Stato Operativo Interface: router-1-2

eth1

UP

eth2

DOWN

lo

UP

lo1

UP

Interface: router-core-2

Stato Amministrativo Interface: router-core-2

eth1

UP

eth2

UP

eth3

DOWN

eth4

UP

lo

UP

lo1

UP

Stato Operativo Interface: router-core-2

eth1

UP

eth2

UP

eth3

DOWN

eth4

UP

lo

UP

lo1

UP

Figura 14 - Stato delle interfacce di router-core-2 e router-1-2 successivamente alla disattivazione dell'interfaccia.

2. Perdita dell'adiacenza tra router-core-2 e router-1-2.

Come previsto, viene mostrato in Figura 15 che router-core-2 (OSPF ID 2.2.2.2) non rileva più alcun vicino sull'interfaccia disattiva e anche router-1-2 di conseguenza perde l'adiacenza.

Area 0.0.0.0		
Router: router-1-2 x router-core-2 x		
Stato Adiacenza con Vicini		
router-core-2 / eth1: 172.20.1.2 → 1.1.1.1		FULL
router-core-2 / eth2: 172.20.2.2 → 1.1.1.1		FULL
router-core-2 / eth3: 172.30.122.2 → 4.4.4.4		FULL
router-core-2 / eth4: 172.30.222.2 → 6.6.6.6		FULL
router-1-2 / eth2: 172.30.122.12 → 2.2.2.2		FULL

Figura 15 - Vicini visti da router-core-2 e router-1-1 prima e dopo lo spegnimento dell'interfaccia.

3. Cambiamenti nel rate di ricezione e trasmissione delle interfacce.

In particolare, oltre all'ovvio azzeramento del rate di ricezione (a) e trasmissione (b) sull'interfaccia eth3 di router-core-2, si può osservare in Figura 16 che il traffico in uscita (a) viene reindirizzato verso router-core-1 attraverso eth2.

In router-1-2, direttamente connesso tramite eth2 all'interfaccia spenta, si osserva una diminuzione del rate di trasmissione di eth1 (c). Osservando la topologia della rete (Figura 4), si può supporre che, prima dell'evento, il router fosse una "tappa intermedia" per il traffico proveniente da router-2-2 e diretto verso la rete che ospita il sistema di monitoraggio, mentre successivamente questo sia "deviato" verso router-core-1.

Analogamente, si notano variazioni anche nei grafici di tutti gli altri router non direttamente connessi all'interfaccia disattivata. A titolo di esempio, in figura Figura 16 sono riportati gli andamenti per il router-core-1, che in concordanza con quanto ipotizzato in precedenza, vede un aumento dei pacchetti inoltrati attraverso eth3 (e) e di quelli ricevuti da router-core-2 sull'interfaccia eth2 (f).

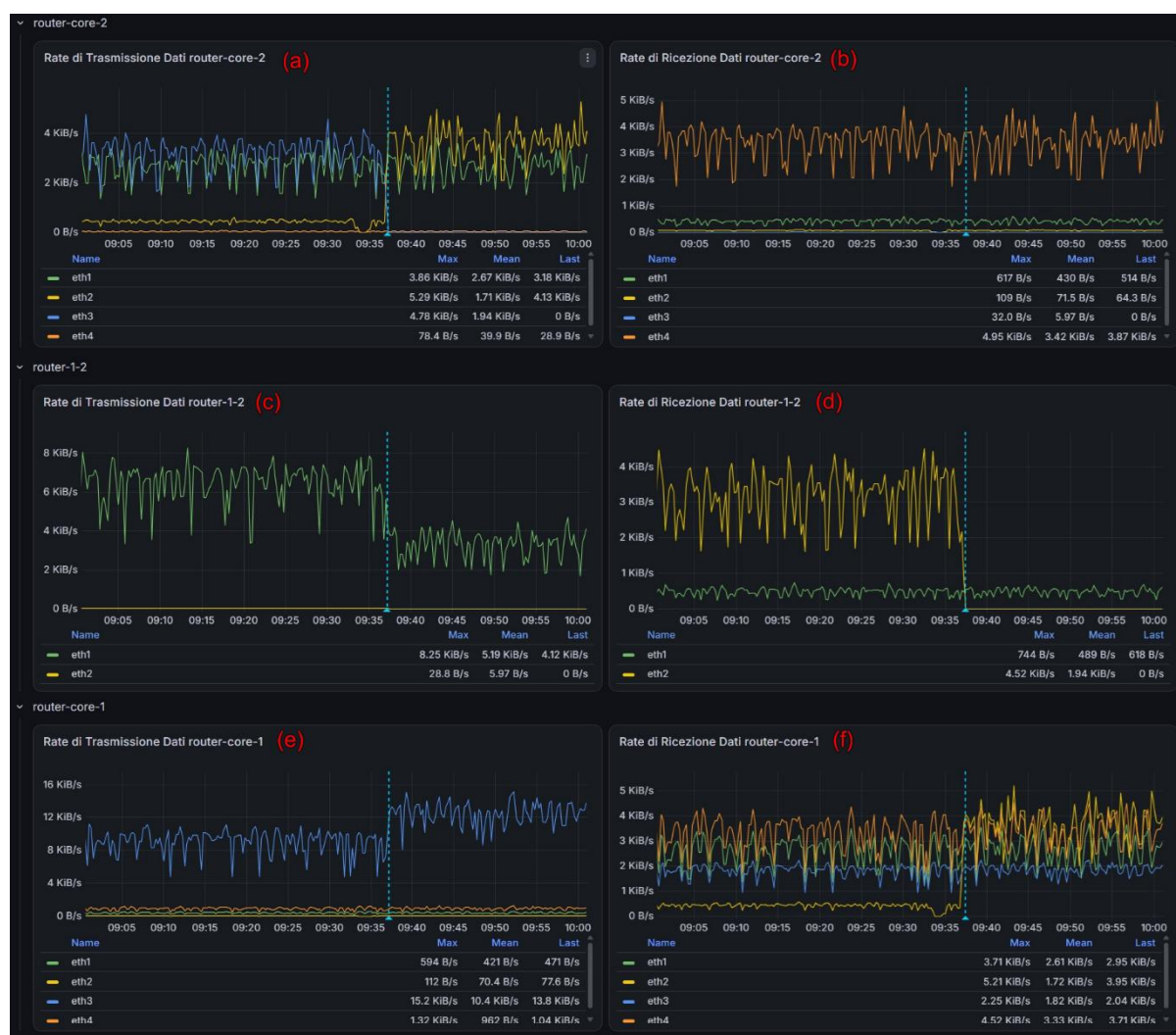


Figura 16 - Variazioni nel rate in trasmissione e ricezione del router-core-2 ((a) e (b)), router-1-2 ((c) e (d)) e router-core-1 ((e) e (f)). La linea azzurra tratteggiata indica approssimativamente l'istante di spegnimento dell'interfaccia eth3 di router-core-2

4. Ricalcolo della topologia OSPF.

In accordo a quanto detto nell'osservazione 3, si nota dalla Figura 17 che si è verificata un'esecuzione dell'algoritmo SPF in tutti i router dell'area 0, e ciò si ripercuote anche nelle variazioni consecutive della Routing Information Base (RIB) e della Forward Information Base (FIB) (Figura 18).

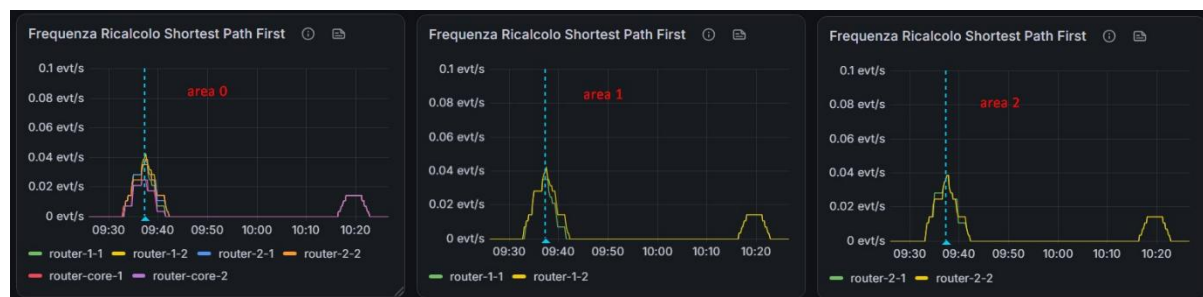


Figura 17 - Picco nella frequenza di ricalcolo di SPF in corrispondenza dell'evento (linea azzurra tratteggiata).



Figura 18 - Variazione nella composizione di RIB e FIB.

2.4.2.2 Test 2: mismatch dell'hello timer

Descrizione	Su un'interfaccia di un router viene configurato un hello timer differente rispetto a quello configurato sull'interfaccia all'altro capo.
Modalità di esecuzione	router-1-1: accesso alla configurazione dell'interfaccia eth2 tramite terminale vtysh e modifica dell'hello timer dal valore di default (10 secondi) a 5 secondi;
Principali effetti attesi	router-core-1: cambiamento dello stato di adiacenza con router-1-1; tutti i router con interfacce nell'area 1 e 0: ricalcolo SPF; scambio di nuovi LSA; - mantenimento della raggiungibilità dell'interfaccia eth2 di router-1-1 da parte di tutti i dispositivi nella rete.

Principali risultati osservati

1. Perdita delle adiacenze.

Come si può osservare dalla Figura 19, non solo si perde lo stato di neighborhood tra router-1-1 e router-core-1, ma questi smettono proprio di vedersi come vicini. Non cambia invece lo stato di router-1-1 come ABR, poichè la sua interfaccia eth2 è comunque configurata con OSPF nell'area 0 e la rete 172.30.111.0/24 continua ad essere pubblicizzata, anche se si suppone che almeno per una parte dei router della rete sia cambiato l'LSA di tipo 2 associato.

Area 0.0.0.0

Router

All

Stato Adiacenza con Vicini

router-1-1 / eth2: 172.30.111.11 → 1.1.1.1

router-core-2 / eth1: 172.20.1.2 → 1.1.1.1

router-core-2 / eth2: 172.20.2.2 → 1.1.1.1

router-core-2 / eth3: 172.30.122.2 → 4.4.4.4

router-core-2 / eth4: 172.30.222.2 → 6.6.6.6

router-2-1 / eth2: 172.30.211.21 → 1.1.1.1

router-1-2 / eth2: 172.30.122.12 → 2.2.2.2

router-core-1 / eth1: 172.20.1.1 → 2.2.2.2

router-core-1 / eth2: 172.20.2.1 → 2.2.2.2

router-core-1 / eth3: 172.30.111.1 → 3.3.3.3

router-core-1 / eth4: 172.30.211.1 → 5.5.5.5

router-2-2 / eth2: 172.30.222.22 → 2.2.2.2

FULL

FULL

FULL

FULL

FULL

FULL

FULL

FULL

FULL

FULL

FULL

ABR

Router ↑

Area Confinante

router-1-1

0.0.0.1

router-1-2

0.0.0.1

router-2-1

0.0.0.2

router-2-2

0.0.0.2

Area 0.0.0.0

Router

All

Stato Adiacenza con Vicini

router-core-2 / eth1: 172.20.1.2 → 1.1.1.1

router-core-2 / eth2: 172.20.2.2 → 1.1.1.1

router-core-2 / eth3: 172.30.122.2 → 4.4.4.4

router-core-2 / eth4: 172.30.222.2 → 6.6.6.6

router-2-1 / eth2: 172.30.211.21 → 1.1.1.1

router-1-2 / eth2: 172.30.122.12 → 2.2.2.2

router-core-1 / eth1: 172.20.1.1 → 2.2.2.2

router-core-1 / eth2: 172.20.2.1 → 2.2.2.2

router-core-1 / eth4: 172.30.211.1 → 5.5.5.5

router-2-2 / eth2: 172.30.222.22 → 2.2.2.2

FULL

FULL

FULL

FULL

FULL

FULL

FULL

FULL

FULL

FULL

FULL

ABR

Router ↑

Area Confinante

router-1-1

0.0.0.1

router-1-2

0.0.0.1

router-2-1

0.0.0.2

router-2-2

0.0.0.2

Figura 19 - Stato delle adiacenze ed elenco degli ABR nell'area 0 prima e dopo il cambiamento dell'hello timer.

2. Cambiamento nel LSDB.

In Figura 20 si può osservare che c'è stato effettivamente un cambiamento nel LSDB di router-1-1. Questo cambiamento non si riflette negli LSDB degli altri router, almeno non quantitativamente. La non conformità del LSDB di un'area tra i router a essa collegati è effettivamente un sintomo di una malconfigurazione in OSPF.

Router ↑	Router LSA (Type 1) Count	Network LSA (Type 2) Count	ABR Summary LSA (Type 3) Count	ASBR Summary LSA (Type 4) Count	Totale
router-1-1	1	0	4	1	6
router-1-2	5	3	10	3	21
router-2-1	5	3	10	3	21
router-2-2	5	3	10	3	21
router-core-1	5	3	10	3	21
router-core-2	5	3	10	3	21

Figura 20 - Composizione degli LSDB dell'area 0 dopo il cambiamento dell'hello timer in router-1-1.

In Figura 21 viene verificato che router-1-1 non ha più LSDB Network (tipo 2) nel database dell'area 0. Ciò si può spiegare notando che router-1-1, non avendo più vicini nell'area 0, non ha modo di sapere quali reti interne sono presenti in quell'area.

<pre>clab@LAPTOP-BSFQRKRR:~/Tesi/demonet\$ docker exec clab-demonet-router-core-1 vtysh -c "sho ip ospf dat network"</pre> <pre> OSPF Router with ID (1.1.1.1) Net Link States (Area 0.0.0.0) LS age: 535 Options: 0x2 : * - - - E - LS Flags: 0x6 LS Type: network-LSA Link State ID: 172.30.122.12 (address of Designated Router) Advertising Router: 4.4.4.4 LS Seq Number: 80000009 Checksum: 0x1cbc Length: 32 Network Mask: /24 Attached Router: 2.2.2.2 Attached Router: 4.4.4.4 </pre>	<pre>clab@LAPTOP-BSFQRKRR:~/Tesi/demonet\$ docker exec clab-demonet-router-1-1 vtysh -c "sho ip ospf dat network"</pre> <pre> OSPF Router with ID (3.3.3.3) Net Link States (Area 0.0.0.0) Net Link States (Area 0.0.0.1) LS age: 487 Options: 0x2 : * - - - E - LS Flags: 0x6 LS Type: network-LSA Link State ID: 172.16.1.2 (address of Designated Router) Advertising Router: 4.4.4.4 LS Seq Number: 8000000b Checksum: 0x7fe4 Length: 32 Network Mask: /24 Attached Router: 2.2.2.2 </pre>
---	--

Figura 21 - Output del comando "show ip ospf database network" in router-core-1 (sinistra) e in router-1-1 (destra).

3. Cambiamento negli andamenti dei rate delle interfacce.

Si ricorda che, come mostrato nello schema di rete in Figura 4, i container di monitoraggio sono raggiungibili solo tramite il router-1-1, coinvolto nel mismatch, perciò tutto il traffico generato dallo scraping delle metriche passa necessariamente da esso. In Figura 22 si può osservare che:

- nel router-1-1 l'andamento del traffico trasmesso (a) di eth1 ed eth2 si inverte. Si suppone che ciò accada perché il router non vedendo più il riceve tutti gli LSA con le rotte verso l'area 0 e 2 dal router-1-2, perciò invia la gran parte del traffico ad esso tramite l'interfaccia eth1. Il rate all'interfaccia eth1 non risulta comunque nullo, perché la sottorete 172.30.111.0/24 è ancora raggiungibile tramite router-core-1. Si ipotizza che il traffico sia generato dall'invio degli hello packets e dalle ICMP echo reply prodotti da ping di test verso l'IP 172.30.111.11 (eth2). Il traffico in ingresso (b) in eth2 è quasi azzerato, mentre aumenta notevolmente il carico sull'interfaccia eth1, da cui riceve attraverso router-1-2 tutte le metriche e i messaggi SNMP destinati all'exporter da parte di tutti i router della rete.
- nel router-core-1 il traffico trasmesso (c) attraverso l'interfaccia eth3, direttamente connessa al router-1-1, si riduce drasticamente, si ipotizza per gli stessi motivi indicati sopra. Di conseguenza, aumenta il traffico trasmesso

dall'interfaccia eth1, di cui probabilmente una buona parte con destinazione nell'area1.

- Il rate in ricezione in router-core-1 (d) diminuisce per tutte le interfacce di tranne eth4, connessa al router-2-1: come si può notare in (f), sembrerebbe che il carico in ricezione si sia spostato sul router-core-2 in conseguenza del cambiamento della topologia OSPF.
- Gli effetti di questo cambio di topologia che fa sì che il router-1-2 diventi il “collo di bottiglia” verso l'area 1 è osservabile anche nell'aumento del rate di trasmissione (e) dell'interfaccia eth3 di router-core-2, collegata al suddetto router. Analogamente, si osserva un aumento generale del traffico in ingresso in tutte le interfacce del router-core-2 tranne che in eth4, collegata al router-2-2.

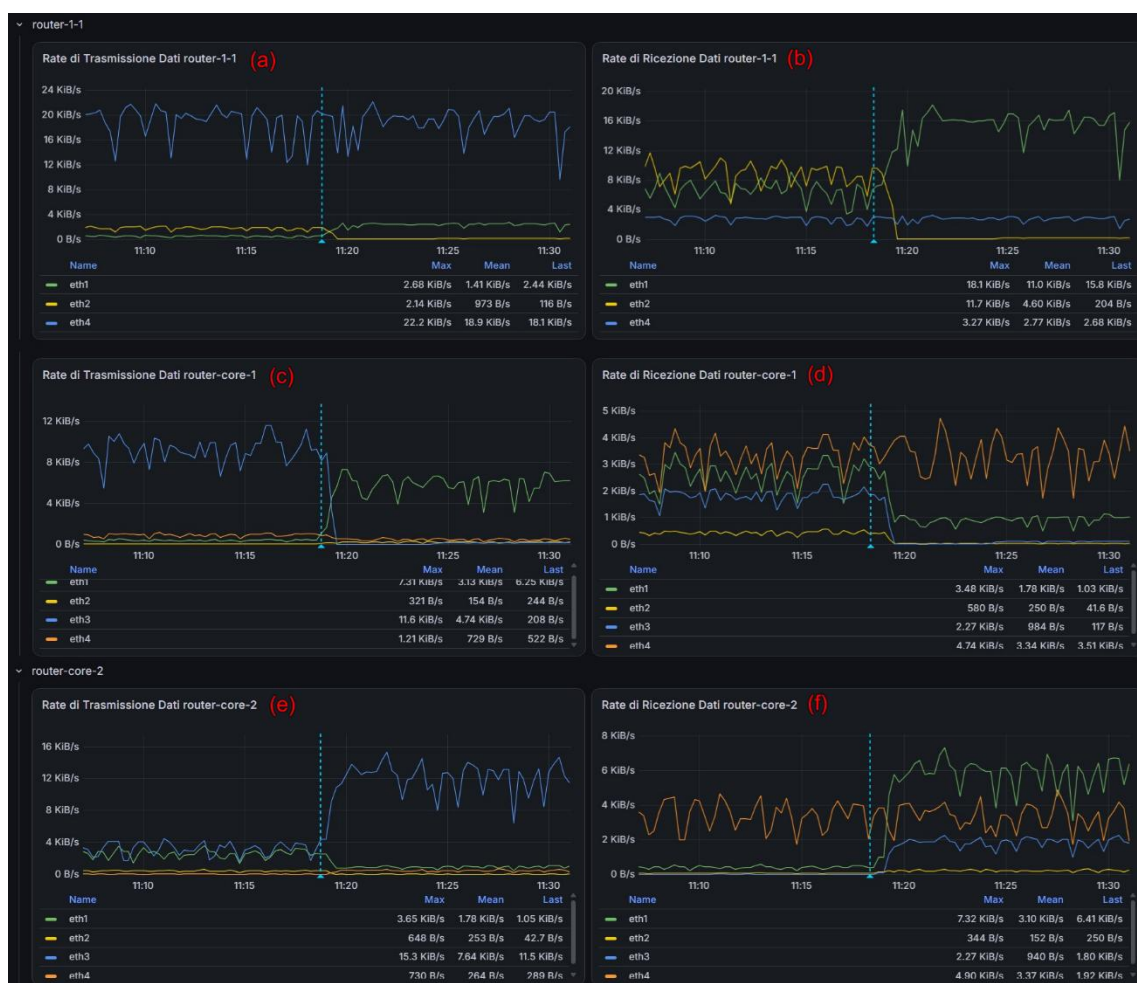


Figura 22 - Cambiamenti nel rate di trasmissione e ricezione di router-1-1 ((a) e (b)), router-core-1 ((c) e (d)) e router-core-2 ((e) e (f)). La linea azzurra tratteggiata indica approssimativamente l'istante in cui è stato modificato l'hello timer.

4. Esecuzione dell'algoritmo SPF.

Si rilevano anche in questo caso nuove esecuzioni di SPF e variazioni in RIB e FIB, similmente a quanto visto nel test precedente.

Considerazioni sul test

Questo test mette in luce un vantaggio del monitoraggio in tempo reale: permette di

rilevare dei problemi che in molti casi non hanno effetti visibili dall'esterno. In questo caso, infatti, la connettività tra tutti i punti della rete è rimasta intatta. Ma il verificarsi di un ulteriore problema o dell'applicazione di una configurazione sbagliata in un altro punto della rete, potrebbe portare all'irraggiungibilità di alcune destinazioni.

3 Conclusioni

3.1 Considerazioni

Prima di questo progetto, mi sono approcciata alla virtualizzazione quasi esclusivamente lavorando con macchine virtuali. Queste sono uno strumento estremamente potente, ma richiedono generalmente del tempo per essere configurate, oltre che una considerevole quantità di risorse, causa l'emulazione completa di hardware e sistema operativo. È stato quindi per me molto interessante scoprire i container, che permettono di avviare in pochi secondi una rete composta anche da molti nodi. Questo rende molto più semplice sperimentare scenari complessi, soprattutto insieme a Containerlab, che permette di realizzare un'infrastruttura di rete tra container con poche righe di configurazione.

La sfida in questo progetto è stato capire come integrare i diversi componenti nel sistema. Grafana e Prometheus mettono a disposizione una documentazione piuttosto esaustiva e sono corredati di esempi pratici guidati, che rendono facile l'apprendimento e la configurazione iniziale. Più complesso è stato invece capire come integrare i vari exporter, in particolare per quanto riguarda SNMP Exporter e il supporto SNMP in FRR. La documentazione disponibile su questo argomento è risultata piuttosto limitata e non è stato semplice reperire esempi completi relativi alla configurazione di SNMP in un ambiente FRR containerizzato. Per superare queste difficoltà è stato necessario confrontarmi con il relatore, che mi ha supportato nella realizzazione dell'immagine utilizzata nel laboratorio.

Nonostante le difficoltà incontrate, il lavoro svolto è stato estremamente formativo e stimolante. Affrontare problemi reali, ricercare soluzioni e integrare tecnologie differenti mi ha permesso di approfondire competenze che vanno oltre i singoli strumenti utilizzati.

3.2 Possibili sviluppi

Avendo costruito una dashboard che, attraverso l'osservazione attiva, permette di rilevare lo stato della rete pressoché in tempo reale, si può aggiungere un ulteriore livello di automazione introducendo degli alert. Prometheus permette di definire delle regole di alerting tramite un file di configurazione YAML, con un'apposita sintassi.

Il monitoraggio in questo progetto è stato focalizzato sul networking: uno sviluppo interessante sarebbe aggiungere dei nodi che implementino un'applicazione distribuita e osservare le performance della rete sotto un carico di traffico elevato.

4 Bibliografia

- [1] Dodin, R. (n.d.). *Containerlab - topology definition*. Containerlab.Dev. Retrieved October 7, 2026, from <https://containerlab.dev/manual/topo-def-file/>
- [2] *FRRouting user guide — FRR latest documentation*. (2018). Frrouting.Org. <https://docs.frrouting.org/en/latest/index.html>
- [3] *GitHub - tynany/frr_exporter: Prometheus exporter for free range routing*. (2026). GitHub. https://github.com/tynany/frr_exporter
- [4] *Google/cadvisor*. (2021, March 16). GitHub. <https://github.com/google/cadvisor>
- [5] Grafana. (2020). *Grafana documentation*. Grafana Labs. Grafana Labs. <https://grafana.com/docs/grafana/latest/>
- [6] Prometheus. (2012). *Overview | prometheus*. Prometheus.io. <https://prometheus.io/docs/introduction/overview/>
- [7] *Prometheus/snmp_exporter*. (2021, May 6). GitHub. https://github.com/prometheus/snmp_exporter

5 Allegati

5.1 Configurazioni

5.1.1 Allegato 1: Topology file

```
1 name: demonet
2
3 topology:
4   defaults:
5     kind: linux
6
7   groups:
8     frr_router:
9       image: frr-snmp:10.7.1
10      network-mode: none
11      binds:
12        - ./frr-configs/shared/daemons:/etc/frr/daemons:ro
13        - ./frr-configs/shared/snmpd.conf:/etc/frr/snmpd.conf:ro
14        - ./frr-configs/shared/vtysh.conf:/etc/frr/vtysh.conf:ro
15        - ./frr-configs/__clabNodeName__:/etc/frr/
16        - ./frr_exporter/frr_exporter:/bin/frr_exporter
17
18   nodes:
19     #=====
20     # Router
21     #=====
22     # Area 0
23     router-core-1:
24       group: frr_router
25     router-core-2:
26       group: frr_router
27     # Area 1
28     router-1-1:
29       group: frr_router
30     router-1-2:
31       group: frr_router
32     # Area 2
33     router-2-1:
34       group: frr_router
35     router-2-2:
36       group: frr_router
37
38     #=====
39     # Switches
40     #=====
41     switch1:
42       kind: bridge
43
44     #=====
45     # Prometheus
46     #=====
47     prometheus:
48       kind: linux
```

```

49     image: prom/prometheus:latest
50     ports:
51       - 9090:9090
52     binds:
53       - ./prometheus/config:/etc/prometheus/
54       - ./prometheus/data:/prometheus/
55     restart-policy: "no"
56     cmd: |
57       --config.file=/etc/prometheus/prometheus.yml
58       --enable-feature=promql-experimental-functions
59       --log.level=debug
60     exec:
61       - ip addr add 172.40.11.2/24 dev eth1
62       - ip route replace default via 172.40.11.1 dev eth1
63       - ip route add 172.18.224.0/20 via 172.20.20.1 dev eth0
64
65     #=====
66     # SNMP Exporter
67     #=====
68     snmp-exporter:
69       kind: linux
70       image: prom/snmp-exporter:latest
71       ports:
72         - 9116:9116
73       binds:
74         - ./snmp-exporter/snmp.yml:/etc/snmp_exporter/snmp.yml:ro
75       exec:
76         - ip addr add 172.40.11.3/24 dev eth1
77         - ip route replace default via 172.40.11.1 dev eth1
78         - ip route add 172.18.224.0/20 via 172.20.20.1 dev eth0
79
80     #=====
81     # cAdvisor
82     #=====
83     cadvisor:
84       kind: linux
85       image: gcr.io/google/cadvisor:0.60.5
86       binds:
87         - /:/rootfs:ro
88         - /var/run:/var/run:ro
89         - /sys:/sys:ro
90         - /var/lib/docker:/var/lib/docker:ro
91       devices:
92         - /dev/kmsg
93       ports:
94         - 8080:8080
95       cmd: |
96         --docker_only=true
97         --housekeeping_interval=15s
98         --store_container_labels=false
99         --
100     disable_metrics=percpu,sched,tcp,udp,diskIO,hugetlb,referenced_memory
101     exec:
102       - ip addr add 172.40.11.4/24 dev eth1
103       - ip route replace default via 172.40.11.1 dev eth1
104       - ip route add 172.18.224.0/20 via 172.20.20.1 dev eth0
105
106     #=====

```

```

107 # Grafana
108 #=====
109 grafana:
110     kind: linux
111     image: grafana/grafana:latest
112     ports:
113         - 3000:3000
114     binds:
115         - ./grafana/:/var/lib/grafana/
116     exec:
117         - ip addr add 172.40.11.5/24 dev eth1
118         - ip route replace default via 172.40.11.1 dev eth1
119         - ip route add 172.18.224.0/20 via 172.20.20.1 dev eth0
120
121 #=====
122 # Links
123 #=====
124 links:
125     # routers
126     # a0
127     - endpoints: ["router-core-1:eth1", "router-core-2:eth1"]
128     - endpoints: ["router-core-1:eth2", "router-core-2:eth2"]
129     - endpoints: ["router-core-1:eth3", "router-1-1:eth2"]
130     - endpoints: ["router-core-1:eth4", "router-2-1:eth2"]
131     - endpoints: ["router-core-2:eth3", "router-1-2:eth2"]
132     - endpoints: ["router-core-2:eth4", "router-2-2:eth2"]
133     # a1
134     - endpoints: ["router-1-1:eth1", "router-1-2:eth1"]
135     - endpoints: ["router-1-1:eth4", "switch1:port1"]
136     # a2
137     - endpoints: ["router-2-1:eth1", "router-2-2:eth1"]
138     # monitoring
139     - endpoints: ["prometheus:eth1", "switch1:port2"]
140     - endpoints: ["snmp-exporter:eth1", "switch1:port3"]
141     - endpoints: ["cadvisor:eth1", "switch1:port4"]
142     - endpoints: ["grafana:eth1", "switch1:port5"]
143     # router loopback interfaces
144     - type: dummy
145       endpoint:
146         node: router-core-1
147         interface: lol
148     - type: dummy
149       endpoint:
150         node: router-core-2
151         interface: lol
152     - type: dummy
153       endpoint:
154         node: router-1-1
155         interface: lol
156     - type: dummy
157       endpoint:
158         node: router-1-2
159         interface: lol
160     - type: dummy
161       endpoint:
162         node: router-2-1
163         interface: lol
164     - type: dummy

```

165	endpoint:
166	node: router-2-2
167	interface: lol

5.1.2 Allegato 2: Configurazioni Prometheus

```
1 global:
2   scrape_interval: 15s
3
4 rule_files:
5   - recorded_rules.yml
6
7 scrape_configs:
8
9   #=====
10  # Prometheus
11  #=====
12  - job_name: 'prometheus'      # self-scrape
13    static_configs:
14      - targets: ['localhost:9090']
15
16  #=====
17  # FRR Exporter
18  #=====
19  - job_name: 'frr'
20    static_configs:
21      - targets: ['192.168.0.1:9342']
22        labels: {hostname: 'router-core-1', group: 'router'}
23      - targets: ['192.168.0.2:9342']
24        labels: {hostname: 'router-core-2', group: 'router'}
25      - targets: ['192.168.1.1:9342']
26        labels: {hostname: 'router-1-1', group: 'router'}
27      - targets: ['192.168.1.2:9342']
28        labels: {hostname: 'router-1-2', group: 'router'}
29      - targets: ['192.168.2.1:9342']
30        labels: {hostname: 'router-2-1', group: 'router'}
31      - targets: ['192.168.2.2:9342']
32        labels: {hostname: 'router-2-2', group: 'router'}
33
34
35  #=====
36  # cAdvisor
37  #=====
38  - job_name: 'cadvisor'
39    static_configs:
40      - targets: ['172.40.11.4:8080']      # IP del container snmp-
41  exporter                                # sulla rete 172.40.11.0/24
42                                          labels: {group: 'exporter'}
43
44
45  #=====
46  # SNMP Exporter
47  #=====
48  - job_name: 'snmp-exporter'
49    static_configs:
50      - targets: ['172.40.11.3:9116']      # IP del container snmp-exporter
51                                          # sulla rete 172.40.11.0/24
52      labels: {group: 'exporter'}
53      #metriche SNMP sulle interfacce di router1 e router2
54  - job_name: 'snmp-network-devices'
55    scrape_interval: 15s
```

```

56 scrape_timeout: 10s
57 static_configs:
58   - targets: ['192.168.0.1']
59     labels: {hostname: 'router-core-1', group: 'router'}
60   - targets: ['192.168.0.2']
61     labels: {hostname: 'router-core-2', group: 'router'}
62   - targets: ['192.168.1.1']
63     labels: {hostname: 'router-1-1', group: 'router'}
64   - targets: ['192.168.1.2']
65     labels: {hostname: 'router-1-2', group: 'router'}
66   - targets: ['192.168.2.1']
67     labels: {hostname: 'router-2-1', group: 'router'}
68   - targets: ['192.168.2.2']
69     labels: {hostname: 'router-2-2', group: 'router'}
70 metrics_path: /snmp
71 params:
72   module: [if_mib, ip_mib, ospf, system]
73   auth: [public_v2]
74 relabel_configs:
75   #The SNMP Exporter needs the target as a parameter
76   - source_labels: [__address__]
77     target_label: __param_target
78   #Keep the original target as a label
79   - source_labels: [__param_target]
80     target_label: instance
81   #Point the scrape at the SNMP Exporter service
82   - target_label: __address__
83     replacement: 172.40.11.3:9116

```

5.1.3 Allegato 3: Recording rules

```
1 groups:
2   - name: ospf
3     interval: 60s
4     rules:
5
6     # Associa l'hostname di un router al suo ID OSPF
7     # Labels: hostname, instance, job, ospfRouterId
8     - record: hostname:ospf_router_id
9       expr: |
10         label_replace(sysName, "hostname", "$1", "sysName", "(.*)") *
11         on (instance) group_right(hostname)
12           ospfRouterId
13
14     # Associa lo stato dei vicini nell'area OSPF in cui si trovano
15     # Labels: ospfRouterId, area, group, hostname, iface, instance,
16 job, local_address, neighbor
17     # Value: 1=Down, 2=Init, 3=2-Way, 4=ExStart, 5=Exchange, 6=Loading,
18 7=Full
19     - record: ospf_neighbor_state:ospf_area
20       expr: |
21         ospfRouterId * on (hostname)
22         group_right(ospfRouterId)
23         frr_ospf_neighbor_state * on(instance, iface)
24         group_left(area)
25         (frr_ospf_neighbors * 0 + 1)
26
27     # Calcola il rate di ricezione dei LS divisi per tipo e per router,
28 basandosi sul numero variazioni nel numero di sequenza di un LS.
29     # Labels: ospfRouterId, ospfLsdbType
30     # Value: rate calcolato su un intervallo di 5 minuti
31     - record: ospf_ls_rate_over_5m:per_area:per_type:per_router
32       expr: |
33         sum by (ospfLsdbType, hostname, ospfLsdbAreaId)
34         ( changes(ospfLsdbSequence[5m])
35         ) / 300
36
37     # Restituisce un vettore contenente il numero di tipi diversi
38     # di LS nel LSDB, per ogni router e area
39     - record: ospf_area_lsdb:per_router_count
40       expr: |
41         ospfRouterId * on(instance)
42         group_right(hostname)
43         ospfLsdbAdvertisement or (ospfRouterId * 0)
44
45     # Restituisce il numero di volte in cui c'è stato
46     # un cambiamento nella relazione col vicino o di errori
47     # Labels: hostname, heighbor, ospfNbrIpAddr
48     - record: ospf_neighbor_events:neighbor_hostname
49       expr: |
50         label_replace(
51         label_replace(ospfIfAdminStat, "ospfNbrIpAddr", "$1",
52 "ospfIfIpAddress", "(.*)")
53         ), "neighbor", "$1", "hostname", "(.*)")
54         * on (ospfNbrIpAddr)
55         group_right(neighbor) ospfNbrEvent
```